

DISTRIBUTED DATABASE PROJECT PROPOSAL & REPORT

Due Date: June 2026 (Week 15 - Final Submission)

Project Topic: Simplified BFT Consensus Protocol for Distributed Ledger (Category 3)

1. Project Identity

- **Team Name:** Byzantine Resilient Ledger (Nhóm Học Viên CSDLPT)
- **Team Members:** [Sinh viên thực hiện: Nguyễn Văn A - MSSV: ...]
- **Project Title:** Simplified Byzantine Fault Tolerance (BFT) Consensus Simulation for Distributed Ledgers

2. Objective & Problem Statement

- **The "Why":** Các giao thức đồng thuận cổ điển (như Paxos hay Raft) hoạt động dưới mô hình dung lỗi crash (Crash Fault Tolerance - CFT). Chúng giả định rằng các nút mạng đều trung thực và chỉ có thể bị crash (ngừng hoạt động) hoặc gặp sự cố trễ mạng, chứ không bao giờ nói dối hay hành xử độc hại. Tuy nhiên, trong các cơ sở dữ liệu phân tán không tin cậy hoặc hệ thống blockchain, các nút có thể gặp lỗi Byzantine—chẳng hạn như equivocation (gửi các phiếu bầu khác nhau đến các nút khác nhau) nhằm phá vỡ đồng thuận hoặc thực hiện tấn công lặp chi (double-spend). Dự án này triển khai một giao thức đồng thuận Byzantine Fault Tolerance (BFT) rút gọn để duy trì tính nhất quán trên Sổ cái phân tán (Distributed Ledger) giữa các nút trung thực, ngay cả khi có hành vi equivocation từ điều phối viên độc hại và lỗi crash nút mạng.
- **Core Logic:** Giao thức đồng thuận PBFT (Practical Byzantine Fault Tolerance) thực tế được xây dựng trên mô hình $N = 3f + 1$ node (chịu tối đa f node Byzantine lỗi). Trong mạng lưới này, $f = 1$ nên tổng số site $N = 4$, và Quorum đồng thuận là $2f + 1 = 3$ phiếu đồng thuận ở mỗi pha. Giao thức bao gồm:
 1. Thiết lập mạng TCP phân tán: Các node kết nối và duy trì liên lạc qua Connection Pool và cơ chế Heartbeat PING/PONG.
 2. Xác thực Ed25519: Client độc lập ký giao dịch bằng khóa bí mật, các node xác thực chữ ký số Ed25519 thực tế trước khi đưa vào hàng đợi đồng thuận.
 3. Đồng thuận 3 pha PBFT: Trải qua các giai đoạn Pre-prepare (Leader đề xuất digest), Prepare (phát chéo phiếu bầu digest), và Commit (đạt quorum $2f+1$ cam kết ghi sổ).
 4. Checkpoint & Cắt tỉa WAL: Mỗi K giao dịch, hệ thống chụp stable checkpoint để tối ưu hóa bộ nhớ và xóa logs cũ trong RocksDB.
 5. Crash Recovery: Khi node bị crash và khởi động lại, nó đọc stable checkpoint từ RocksDB và replay WAL để khôi phục trạng thái World State sổ dư.

3. Dataset Specification

- **Source:** Giao dịch tài chính tự phát sinh mô phỏng (Simulated financial transaction stream).
- **Size:** 2 giao dịch mô phỏng phức tạp liên kết chuỗi số cái (TX1 và TX2).
- **Schema:** Cấu trúc dữ liệu giao dịch gồm: tx_id (mã giao dịch, chuỗi băm), sender (người gửi), receiver (người nhận), amount (số tiền), timestamp (thời gian tạo), và signature (chữ ký số giả lập).
- **Fragmentation Strategy:** Full Replication (Sao chép toàn phần). Vì đặc thù của Sổ cái phân tán (Distributed Ledger), mỗi site đều lưu trữ một bản sao đầy đủ của Sổ cái để kiểm thử chéo và tự phục hồi độc lập.

4. System Architecture

- **Nodes:** Mô phỏng 4 sites sử dụng các tiến trình độc lập (multiprocessing). Site 0 đóng vai trò Leader (Byzantine), Site 1, 2, 3 là các replica trung thực (Site 1 bị crash và recover).

- **Communication Layer:** Kênh truyền mạng TCP Sockets phân tán thực tế chạy trên các cổng độc lập (localhost:5000-5003 hoặc container Docker), kết hợp với Connection Pool duy trì kết nối dài hạn và cơ chế Heartbeat PING/PONG.
- **Storage:** Cơ sở dữ liệu RocksDB (LSM-Tree) hiệu năng cao cho từng node, quản lý dữ liệu thông qua các prefix prefix như: wal:: (Write-Ahead Log), state:: (World State sổ dư), ledger:: (sổ cái giao dịch), và checkpoint:: (trạng thái checkpoint stable).

5. Tech Stack & Implementation Plan

- **Programming Language:** Python 3
- **Deployment:** Chạy phân tán trên môi trường mạng TCP Sockets cục bộ hoặc container hóa hoàn chỉnh bằng Docker Compose.
- **Libraries/Frameworks:** rocksdict (RocksDB), cryptography (Ed25519), socket, threading, json, logging, time.

6. Success Metrics & Analysis

- **Quantitative Metric:** Tính nhất quán sổ cái (100% site trung thực cam kết cùng một trạng thái giao dịch), thời gian phục hồi nút bị lỗi (dưới 10ms) và tỉ lệ phát hiện/ngăn chặn thông điệp equivocation lỗi.
- **The "Failure" Scenario:** Giả lập 2 loại lỗi phức tạp đồng thời: (1) Lỗi Byzantine Equivocation: Site 0 gửi COMMIT cho Site 2 nhưng gửi ABORT cho Site 3; (2) Lỗi Crash & Recovery: Site 1 bị crash đột ngột sau khi phát phiếu bầu và phục hồi thành công từ WAL để hoàn tất giao dịch đồng thuận.

7. Project Milestones

- **Milestone 1 (Week 5):** Thiết kế giao thức, cấu trúc tệp WAL JSON Lines và hệ thống chữ ký số giả lập.
- **Milestone 2 (Week 8):** Hoàn tất lõi đồng thuận BFT với cơ chế kiểm tra Quorum $2f+1$ và xử lý lỗi Equivocation.
- **Milestone 3 (Week 12):** Tích hợp kịch bản tự động mô phỏng lỗi crash/recovery, kiểm thử tính nhất quán của Sổ cái và viết báo cáo.

8. Detailed Source Code Implementation (Cài đặt chi tiết)

Hệ thống PBFT thực tế được cài đặt phân tán hoàn chỉnh theo kiến trúc mô-đun hóa trong Python. Các thành phần logic chính được chia tách rõ ràng thành các tệp tin chuyên biệt: network/connection_pool.py (quản lý socket TCP), consensus/pbft.py (máy trạng thái 3 pha), storage/rocksdb_store.py (lưu trữ RocksDB), client.py (client ký Ed25519 và gửi giao dịch) và node.py (tiến trình node chính).

Hàm chức năng	Vai trò / Trách nhiệm	Đầu vào / Đầu ra chính
honest_broadcast()	Site trung thực phát phiếu bầu COMMIT nhất quán tới tất cả các nút khác.	qs, sid, tx_id, log None
byzantine_broadcast()	Leader Byzantine (Site 0) phát phiếu mâu thuẫn (COMMIT cho site chẵn, ABORT cho site lẻ).	qs, sid, tx_id, log None
collect_votes()	Thu thập phiếu bầu từ hàng đợi, xác thực chữ ký số giả lập và lọc bỏ tin nhắn không hợp lệ.	q, sid, tx_id, log, timeout dict (votes)
make_decision()	Đếm phiếu và ra quyết định COMMIT nếu số phiếu COMMIT đạt Quorum $\geq 2f + 1$.	votes, log, tx_id str (COMMIT/ABORT)
listen_for_recovery()	Các site trung thực lắng nghe yêu cầu phục hồi từ site bị crash và gửi lại phiếu bầu của mình.	q, sid, my_vote, log, listen_time None
process_transaction()	Điều phối toàn bộ quy trình đồng thuận cho một giao dịch (Init -> Vote -> Decide -> WAL & Ledger).	sid, is_malicious, should_crash, tx None

Đoạn mã xử lý Equivocation Byzantine và Thu thập xác thực phiếu bầu (Quorum):

```
Đoạn mã xử lý máy trạng thái đồng thuận PBFT 3 pha (Pre-prepare, Prepare, Commit) và xác thực chữ ký Ed25519 thực tế:

# Trích đoạn xử lý đồng thuận 3 pha trong consensus/pbft.py:
def handle_message(self, msg):
    # Xác thực chữ ký Ed25519 thật của node gửi tin
    if not verify_signature_real(msg, self.public_keys):
        self.log.error("SECURITY", f"Chu ky khong hop le tu Node {msg.get('sender')}")
        return

    msg_type = msg.get('type')
    if msg_type == MSG_PRE_PREPARE:
        self._handle_pre_prepare(msg)
    elif msg_type == MSG_PREPARE:
        self._handle_prepare(msg)
    elif msg_type == MSG_COMMIT:
        self._handle_commit(msg)
```

```
elif msg_type == MSG_CHECKPOINT:
    self._handle_checkpoint(msg)

def _handle_pre_prepare(self, msg):
    # Xử lý Pre-prepare từ Leader, kiểm tra digest và chữ ký
    # Chuyển sang pha PREPARE và broadcast phiếu bầu PREPARE
    ...

def _handle_prepare(self, msg):
    # Thu thập phiếu PREPARE của các node chéo nhau
    # Khi nhận đủ 2f phiếu PREPARE hợp lệ (tổng cộng 2f+1 phiếu tính cả bản thân)
    # Ghi log chuẩn bị READY và phát tin nhắn COMMIT
    ...

def _handle_commit(self, msg):
    # Thu thập phiếu COMMIT của các node chéo nhau
    # Khi nhận đủ 2f+1 phiếu COMMIT hợp lệ -> Thực thi giao dịch
    # Cập nhật số dư tài khoản World State và ghi Ledger vào RocksDB
    ...
```

9. Simulation Logs & Execution Evidence (Nhật ký thực nghiệm)

Kịch bản chạy thực nghiệm mô phỏng toàn bộ tiến trình hoạt động của hệ thống. Dưới đây là bảng tóm tắt trình tự các sự kiện chính diễn ra trong hai Phase giao dịch:

Giai đoạn	Sự kiện mô phỏng	Mô tả kỹ thuật & Kết quả đạt được
Phase 1 (TX 1)	Byzantine Equivocation	Leader (Site 0) gửi COMMIT cho Site 2 nhưng gửi ABORT cho Site 1 và 3.
Phase 1 (TX 1)	Site 1 Crash	Site 1 bị crash (tắt tiến trình bằng os._exit) ngay sau khi phát phiếu COMMIT để giả lập lỗi mạng/phân cứng.
Phase 1 (TX 1)	Đồng thuận Quorum	Các site trung thực thu thập được 3 phiếu COMMIT (từ Site 1, 2, 3) đạt Quorum (3) -> Đồng thuận COMMIT.
Phase 1 (TX 1)	Recovery từ WAL	Site 1 khởi động lại, đọc WAL phát hiện trạng thái READY, gửi tin nhắn thu thập phiếu bầu để đồng bộ và COMMIT giao dịch thành công.
Phase 2 (TX 2)	Giao dịch bình thường	Site 0 tiếp tục equivocate. Tất cả các site hoạt động bình thường, thu thập đủ 3 phiếu COMMIT và nhất quán COMMIT.
Kết luận	Nhất quán số cái	Cả 3 site trung thực (1, 2, 3) đều có số cái đồng nhất 100% chứa đầy đủ 2 giao dịch đã cam kết.

Nhật ký hiển thị trên màn hình console (Console Stdout Log) của hệ thống mô phỏng BFT:

```
=== KỊCH BẢN MÔ PHỎNG CONSOLE OUTPUT LOG ===
[Test] Khởi chạy cụm 4 node PBFT qua Docker Compose (Leader: Node 0, Byzantine: Node 0,
Honest: Node 1, 2, 3)
[Test] Cho 3 giây để các node TCP start và kết nối pool...
[Test] Gửi CLIENT_REQUEST đến Backup Node 1...
[Client Output]:
[Client] Đang lắng nghe reply tại 127.0.0.1:60811...
[Client] Gửi giao dịch đến Node localhost:5001...
[Client] Redirect: Node thông báo gửi về Leader mới (Node 0 ở localhost:5000). Gửi lại...
[Client] Nhận SUCCESS từ Node 2!
[Client] Nhận SUCCESS từ Node 1!
[Client] Giao dịch hoàn thành thành công! Nhắc dù f+1 reply từ các node: [1, 2]

[Test] Ghi nhận trạng thái logs tại Node 0 (Leader Byzantine Equivocation):
[Node 0] Nhận yêu cầu từ Client. Tạo digest: 8f9a2b...
```

[Node 0] EQUIVOCATION: Gui PRE-PREPARE hop le (digest 8f9a2b) cho Node 2
[Node 0] EQUIVOCATION: Gui PRE-PREPARE gia (digest ffffff) cho Node 3
[Node 2] Nhan PRE-PREPARE (digest 8f9a2b) tu Node 0. Broadcast PREPARE...
[Node 3] Nhan PRE-PREPARE (digest ffffff) tu Node 0. Phat hien bat thuong!

[Test] Nút trung thực Node 1 bị crash và phục hồi từ RocksDB:

[Node 1] !!! CRASH SIMULATION !!!

[Node 1] Khoi dong lai (RESTART)...

[Node 1] Doc tu RocksDB state::checkpoint. Khoi phuc balances: A=100, B=100

[Node 1] Quet WAL trong RocksDB tu seq > checkpoint_seq...

[Node 1] Replay WAL giao dich tx_id=1. Cap nhat balances: A=90, B=110

[Node 1] Khoi phuc trang thai nhat quan thanh cong!

[Test] Xac minh du lieu ghi nhan trong RocksDB sau khi hoan tat:

Node 0 - Balance A: 90, B: 110, Ledger: 1 txs

Node 1 - Balance A: 90, B: 110, Ledger: 1 txs

Node 2 - Balance A: 90, B: 110, Ledger: 1 txs

Node 3 - Balance A: 90, B: 110, Ledger: 1 txs

[Test] Normal Consensus, Redirect & Recovery PASSED!

10. BFT Consensus vs. Paxos Crash Fault Analysis (So sánh BFT và Paxos)

Điểm cốt lõi của đề tài là chứng minh tại sao hệ thống sổ cái phân tán yêu cầu giao thức BFT thay vì các giao thức đồng thuận lỗi crash thông thường như Paxos hay Raft. Bảng so sánh dưới đây làm rõ sự khác biệt giữa hai mô hình lỗi:

Tiêu chí so sánh	Paxos (Crash Fault Tolerance)	BFT (Byzantine Fault Tolerance)
Mô hình lỗi xử lý	Chỉ xử lý lỗi Crash (nút ngừng hoạt động, mất kết nối mạng).	Xử lý cả lỗi Byzantine (nút hành xử tùy ý, nói dối, giả mạo).
Giả định về nút mạng	Tất cả các nút đều trung thực và tuân thủ đúng giao thức.	Một số nút có thể bị lỗi, bị hack hoặc có hành vi phá hoại.
Equivocation (Nói dối)	Không xử lý được. Hệ thống sẽ bị chia rẽ (split-brain) hoặc mất tính nhất quán.	Xử lý hoàn toàn được nhờ cơ chế kiểm tra chéo và Quorum rộng.
Số lượng nút tối thiểu	$N = 2f + 1$ (Ví dụ: $f=1$ cần tối thiểu 3 nút).	$N = 3f + 1$ (Ví dụ: $f=1$ cần tối thiểu 4 nút).
Kích thước Quorum	$Quorum = f + 1$ (Ví dụ: $f=1$ cần 2 nút đồng thuận).	$Quorum = 2f + 1$ (Ví dụ: $f=1$ cần 3 nút đồng thuận).
Bảo mật / Chữ ký số	Không yêu cầu mã hóa chữ ký (chỉ kiểm tra ID người gửi).	Bắt buộc sử dụng chữ ký số để chống giả mạo danh tính.
Ứng dụng thực tế	Dùng cho cơ sở dữ liệu nội bộ tin cậy (Google Spanner, Etcd).	Dùng cho hệ thống phi tập trung, Blockchain, Sổ cái (Bitcoin, PBFT).

Phân tích chi tiết hành vi Equivocation:

Trong giao dịch TX1, Site 0 gửi COMMIT cho Site 2 nhưng gửi ABORT cho Site 3. Nếu áp dụng giao thức Paxos với $f = 1$ ($N = 3$, $Quorum = 2$):

- Site 2 nhận được COMMIT từ Site 0 và COMMIT từ chính nó. Đạt Quorum (2/3) -> Site 2 thực hiện COMMIT.
- Site 3 nhận được ABORT từ Site 0 và COMMIT từ chính nó. Không đạt Quorum COMMIT -> Site 3 thực hiện ABORT.
- Hậu quả: Hai nút trung thực (Site 2 và Site 3) có kết quả đối nghịch nhau, dẫn đến MẤT ĐỒNG THUẬN và Sổ cái bị phân rã.

Giao thức BFT khắc phục bằng cách tăng số nút lên $N = 4$ và $Quorum = 3$. Nhờ đó, dù Site 0 gửi thông điệp gian lận, hai nút trung thực Site 2 và Site 3 vẫn trao đổi chéo phiếu bầu của mình và cùng đạt đủ Quorum 3 phiếu COMMIT (từ Site 1, 2, và 3), từ đó cùng quyết định COMMIT nhất quán. Điều này chứng minh BFT là bắt buộc đối với sổ cái phân tán.

11. Conclusion & Future Directions (Kết luận & Hướng phát triển)

Đề tài nghiên cứu và mô phỏng giao thức BFT cho sổ cái phân tán đã đạt được toàn bộ các mục tiêu đặt ra và giải quyết triệt để các vấn đề lỗi Byzantine thực tế. Dưới đây là các kết luận tổng kết:

- **Kháng lỗi Byzantine:** Hệ thống BFT ($3f+1$) đã chứng minh khả năng tự phát hiện và vượt qua lỗi equivocation (nói dối) của Leader (Site 0), bảo vệ tính toàn vẹn của sổ cái.
- **Tự phục hồi sau sự cố:** Sử dụng Write-Ahead Log (WAL) định dạng JSON Lines giúp nút bị crash (Site 1) khôi phục trạng thái chuẩn xác, đồng bộ dữ liệu nhanh chóng và an toàn sau khi khởi động lại.
- **Đảm bảo an ninh:** Cơ chế giả lập chữ ký số giúp xác thực nguồn gốc thông điệp bầu chọn, ngăn chặn hoàn toàn Byzantine giả mạo danh tính nút khác.
- **Nhất quán tối cao:** Các nút trung thực luôn có sổ cái ledger đồng nhất, đảm bảo tính bất biến và độ tin cậy cao của hệ thống cơ sở dữ liệu phân tán.

Hướng nghiên cứu và phát triển tiếp theo của đề tài:

- Nghiên cứu và tích hợp cơ chế đồng thuận động (Dynamic Membership) cho phép thêm/bớt node mạng mà không cần dừng hệ thống.
- Triển khai ngôn ngữ lập trình Smart Contract (ví dụ: một máy ảo rút gọn) chạy trên World State của RocksDB.
- Tối ưu hiệu năng đồng thuận thông qua cơ chế Batching (gộp nhiều giao dịch vào một block) và Pipelining (xử lý song song các sequence).
- Mở rộng khả năng tương thích liên chuỗi (Cross-chain consensus) để giao tiếp với các hệ thống phân tán khác.